PROJEKTDOKUMENTATION

# Homeserver mit Pi-hole, Tailscale und Monitoring

Werbeblocker, Reverse Proxy, eigene Cloud und Monitoring auf ausgemusterter Hardware, erreichbar zuhause und unterwegs, ohne offenen Port ins Internet

| Angabe | Wert |
|---|---|
| Autor | Tscheky |
| Ausbildung | IT Netzwerktechnik |
| Stand | 24. Oktober 2026 |
| Repository | github.com/Tscheky0/Home-Server |

# Inhaltsverzeichnis

1 Einleitung ..... 4
1.1 Ziel des Projekts ..... 4
1.2 Ausgangslage ..... 4
1.3 Kennzahlen ..... 4
2 Hardware und Netzwerk ..... 5
3 Teil 1: Grundsystem

## Inhalt

1 Einleitung ..... 4
1.1 Ziel des Projekts ..... 4
1.2 Ausgangslage ..... 4
1.3 Kennzahlen ..... 4
2 Hardware und Netzwerk ..... 5
3 Teil 1: Grundsystem ..... 6
3.1 Ubuntu Server und Netplan ..... 6
3.2 Docker und Portainer ..... 6
3.3 Pi-hole ..... 6
3.4 Tailscale ..... 6
3.5 SSH Härtung ..... 6
4 Teil 2: Erweiterung ..... 7
4.1 Was macht was ..... 7
4.2 Reverse Proxy, Namen und HTTPS ..... 8
4.3 Monitoring mit Prometheus und Grafana ..... 8
4.4 Uptime Kuma und Alarme ..... 8
4.5 Nextcloud ..... 9
4.6 Homepage ..... 9
4.7 Speicherbudget ..... 9
6 Sicherheit ..... 14
7 Fehlersuche: Probleme und Lösungen ..... 15
7.1 Grundsystem ..... 15
7.2 Erweiterung ..... 15
8 Ergebnis ..... 17
9 Was ich gelernt habe ..... 19

..... 6
3.1 Ubuntu Server und Netplan ..... 6
3.2 Docker und Portainer ..... 6
3.3 Pi-hole ..... 6

3.4 Tailscale ..... 6
3.5 SSH Härtung ..... 6
4 Teil 2: Erweiterung ..... 7
4.1 Was macht was ..... 7
4.2 Reverse Proxy, Namen und HTTPS ..... 8
4.3 Monitoring mit Prometheus und Grafana ..... 8
4.4 Uptime Kuma und Alarme ..... 8
4.5 Nextcloud ..... 9
4.6 Homepage ..... 9
4.7 Speicherbudget ..... 9
6 Sicherheit ..... 14
7 Fehlersuche: Probleme und Lösungen ..... 15
7.1 Grundsystem ..... 15
7.2 Erweiterung ..... 15
8 Ergebnis ..... 17
9 Was ich gelernt habe ..... 19

# 1 Einleitung

## 1.1 Ziel des Projekts

Aus einem ausgemusterten PC entsteht ein Homeserver, der im ganzen Haushalt und unterwegs Werbung und Tracking auf DNS Ebene blockt. Im zweiten Schritt wird er zu einem kleinen Rechenzentrum ausgebaut: mit Reverse Proxy und HTTPS, eigener Cloud, Monitoring mit Diagrammen, Ausfallalarmen aufs Handy und einer gemeinsamen Startseite.

Eine feste Vorgabe gilt für das ganze Projekt: **kein einziger Port wird ins Internet geöffnet.** Der Zugriff von außen läuft ausschließlich über das Mesh VPN Tailscale.

## 1.2 Ausgangslage

Der Server hat weder WLAN noch eine Kabelverbindung zum Router. Er hängt per LAN Kabel direkt an einem Windows PC, der seine WLAN Verbindung über die **Windows Internetverbindungsfreigabe (ICS)** weitergibt. Dieser Aufbau hat mehrere echte Netzwerkprobleme verursacht. Ihre Analyse und Lösung bilden den Kern des Projekts.

## 1.3 Kennzahlen

| Kennzahl | Wert |
|---|---|
| Geblockte Domains | 280.282 (StevenBlack und Hagezi Pro) |
| Offene Ports ins Internet | 0 |
| Container | 16 in 8 Stacks |
| Überwachte Dienste | 12 Monitore, 5 Grafana Alarme, Push aufs Handy |
| Hardware | Alter PC, 4 Kerne, 4 GB RAM, 500 GB HDD |
| Fernzugriff | Tailscale, SSH nur mit Ed25519 Schlüssel |
| Speicher | 452 GB System Partition (LVM), Medien für Jellyfin |
| Backup | Jede Nacht um 3:30, Nextcloud Datenbank als SQL Dump, 3 Tage aufbewahrt |

# 2 Hardware und Netzwerk

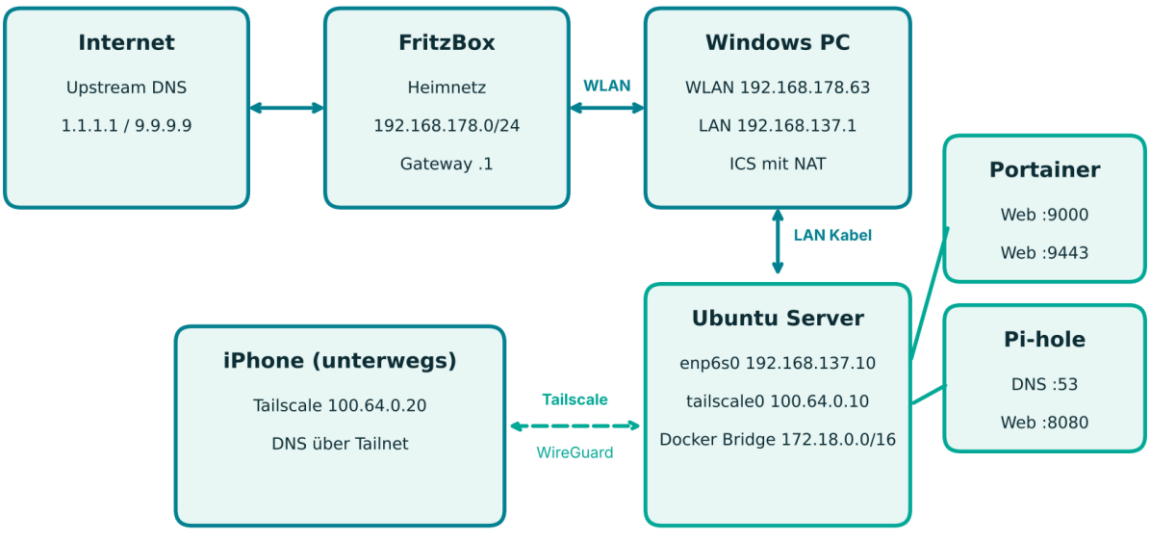



*Abbildung 1: Netzwerkaufbau mit Windows ICS und Tailscale*

| Gerät oder Schnittstelle | Adresse | Rolle |
|---|---|---|
| FritzBox | `192.168.178.1` | Gateway im Heimnetz |
| Windows PC, WLAN | `192.168.178.x` | Uplink ins Heimnetz |
| Windows PC, Ethernet | `192.168.137.1` | Gateway, DHCP und NAT für den Server (ICS) |
| Ubuntu Server, enp6s0 | `192.168.137.10/24` | statisch per Netplan |
| Ubuntu Server, tailscale0 | `100.x.x.x` | DNS Server und Proxy für das Tailnet |
| Docker Netz proxy | `172.x.0.0/16` | gemeinsames Netz aller Weboberflächen |

> Tailscale Adressen, MAC Adressen und Kontonamen sind in dieser Dokumentation anonymisiert.

# 3 Teil 1: Grundsystem

## 3.1 Ubuntu Server und Netplan

Der Server bekommt per Netplan eine feste Adresse im ICS Netz. Gateway und DNS zeigen auf den Windows PC.

```
sudo chmod 600 /etc/netplan/50-cloud-init.yaml
sudo netplan apply
ip route && ping -c 3 1.1.1.1
```

## 3.2 Docker und Portainer

Jeder Dienst läuft in einem eigenen Container. Portainer stellt eine Weboberfläche bereit, um Container und Stacks zu verwalten. Weil Portainer Zugriff auf den Docker Socket und damit volle Kontrolle über den Server hat, ist er nur an die LAN Adresse gebunden.

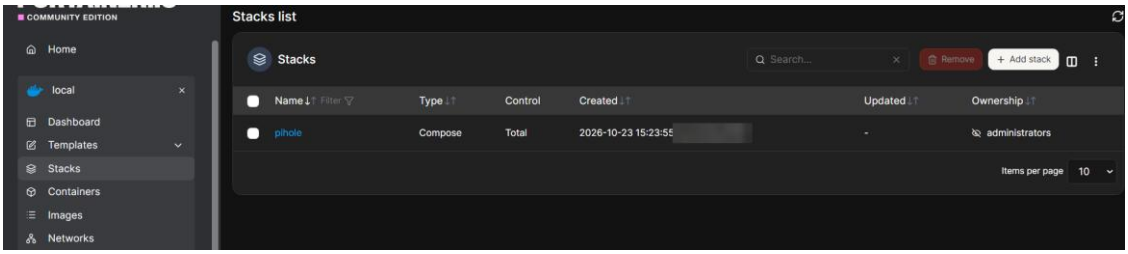



*Abbildung 2: Stacks in Portainer*

## 3.3 Pi-hole

Pi-hole ist der DNS Server des Netzes. Er übersetzt Namen in Adressen und beantwortet Anfragen an bekannte Werbe und Tracking Domains mit einer leeren Antwort. Port 53 ist gezielt nur auf der LAN und der Tailscale Adresse gebunden, damit der DNS Server nie aus fremden Netzen erreichbar ist.

## 3.4 Tailscale

Tailscale baut ein Mesh VPN auf WireGuard Basis auf und funktioniert auch hinter CGNAT, ganz ohne Portfreigabe. In der Admin Konsole ist der Server als globaler DNS Server eingetragen. Dadurch nutzen alle Handys im Tailnet den Pi-hole, auch unterwegs über mobile Daten.

## 3.5 SSH Härtung

Der Login erfolgt nur noch mit einem Ed25519 Schlüssel. Passwort Login und Root Login sind abgeschaltet. Der Schlüssel wurde erst übertragen und getestet, bevor der Passwort Login deaktiviert wurde, um sich nicht auszusperren.

```
sudo cp config/ssh/00-hardening.conf /etc/ssh/sshd_config.d/
sudo sshd -t && sudo systemctl restart ssh
```

# 4 Teil 2: Erweiterung

Vor der Umsetzung wurden alle Entscheidungen festgelegt und mit Begründung in `docs/ERWEITERUNG.md` dokumentiert. Die wichtigsten davon:

| Thema | Entscheidung | Begründung |
|---|---|---|
| Struktur | Jeder Dienst ein eigener Stack unter `docker/<name>/` mit `.env.example` | Einzeln startbar und wartbar, gleiches Muster wie Teil 1 |
| Netz | Ein gemeinsames Docker Netz `proxy` für alle Weboberflächen | Der Proxy erreicht Container über ihren Namen, ohne Host Ports |
| Ports | Nur der Proxy öffnet 443 (LAN und Tailscale), Verwaltung 81 nur im LAN | Eine einzige, kontrollierte Eingangstür |
| Namen | Wildcard `address=/home/<IP>` im Pi-hole | Jeder neue Dienst ist sofort unter `.home` erreichbar |
| HTTPS | Eigene CA mit mkcert, alle Namen als SAN | Let's Encrypt ist ohne öffentliche Domain und offene Ports nicht möglich |
| Dashboard | Homepage statt Homarr | Konfiguration als YAML im Repo statt Klicks in einer Datenbank |
| Socket | Homepage sieht Docker nur über einen Socket Proxy | Socket Zugriff ist Root Zugriff, es wird nur Lesen erlaubt |
| RAM | `mem_limit` auf allen neuen Containern | 4 GB RAM gesamt, kein Dienst darf den Server lahmlegen |

## 4.1 Was macht was

| Komponente | Aufgabe |
|---|---|
| Nginx Proxy Manager | Die Eingangstür. Nimmt `https://grafana.home` entgegen und leitet zum richtigen Container weiter. |
| mkcert | Eigene kleine Zertifizierungsstelle, stellt das HTTPS Zertifikat für alle `.home` Namen aus. |
| Uptime Kuma | Der Wachhund. Prüft jede Minute alle Dienste und meldet Ausfälle über ntfy aufs Handy. |
| Nextcloud | Eigene Cloud wie Google Drive, für Dateien, Fotos und Kalender. |
| MariaDB, Redis, Cron | Datenbank, Zwischenspeicher und Hintergrund Jobs für Nextcloud. |
| Prometheus | Das Gedächtnis. Sammelt alle 30 Sekunden Messwerte und speichert sie 15 Tage. |
| Node Exporter | Liefert Werte des Servers: CPU, RAM, Platte, Netzwerk. |
| cAdvisor | Liefert den Verbrauch jedes einzelnen Containers. |
| Pi-hole Exporter | Liefert die Pi-hole Statistik: Anfragen, Blockquote, Clients. |
| Grafana | Der Bildschirm. Zeichnet aus den Prometheus Daten Live Diagramme. |
| Homepage | Startseite mit Kacheln und Live Werten aller Dienste. |
| docker-socket-proxy | Türsteher, über den Homepage nur lesen darf, welche Container laufen. |

## 4.2 Reverse Proxy, Namen und HTTPS

So kommt eine Anfrage vom Handy bei Grafana an:

1. Das Handy fragt Pi-hole nach `grafana.home`. Die Wildcard Regel beantwortet jeden Namen unter `.home` mit der Adresse des Servers.
2. Der Browser baut HTTPS zum Nginx Proxy Manager auf. Das Zertifikat stammt von der eigenen CA, die auf den Geräten als vertrauenswürdig installiert ist.
3. Der Proxy leitet über das interne Docker Netz `proxy` an `grafana:3000` weiter. Grafana selbst hat keinen Port nach außen.

| Adresse | Ziel im Docker Netz | Dienst |
|---|---|---|
| `https://dashboard.home` | `homepage:3000` | Homepage |
| `https://pihole.home` | `pihole:80` | Pi-hole |
| `https://npm.home` | `npm:81` | Nginx Proxy Manager |
| `https://kuma.home` | `uptime-kuma:3001` | Uptime Kuma |
| `https://grafana.home` | `grafana:3000` | Grafana |
| `https://cloud.home` | `nextcloud:80` | Nextcloud |
| `https://portainer.home` | `portainer:9443` | Portainer |

Zertifikat, Proxy Hosts und alle Einstellungen wurden nicht von Hand geklickt, sondern über die REST API von Nginx Proxy Manager angelegt. Dadurch ist der Aufbau reproduzierbar.

## 4.3 Monitoring mit Prometheus und Grafana

Prometheus fragt alle 30 Sekunden vier Ziele ab: sich selbst, den Node Exporter (Server), cAdvisor (Container) und den Pi-hole Exporter. Grafana nutzt Prometheus als Datenquelle. Datenquelle und Dashboards werden beim Start automatisch per **Provisioning** eingerichtet:

- **Node Exporter Full** (ID 1860): CPU, RAM, Platte, Netzwerk und Temperatur
- **cAdvisor Exporter** (ID 14282): Verbrauch pro Container
- **Pi-hole Exporter** (ID 10176): Anfragen, geblockte Domains, Clients

Prometheus hat keinen Login und ist deshalb absichtlich nur intern erreichbar. Der Node Exporter läuft im Host Netzwerk, damit er die echten Netzwerkwerte sieht, lauscht aber nur auf der LAN Adresse.

## 4.4 Uptime Kuma und Alarme

Uptime Kuma prüft jede Minute zehn Ziele. Fällt eines zweimal hintereinander aus, kommt über **ntfy** eine Push Nachricht aufs Handy. Admin, Benachrichtigung, alle Monitore und die öffentliche Status Page legt das Skript `setup-kuma.js` in einem Schritt über die Socket.IO Schnittstelle an.

| Monitor | Art | Ziel |
|---|---|---|
| Server Ping | Ping | `192.168.137.10` |
| Pi-hole DNS | DNS | Auflösung von `google.com` über den Pi-hole |
| Pi-hole Web, Portainer, NPM | HTTP | jeweilige Weboberfläche |

| Monitor | Art | Ziel |
|---|---|---|
| Nextcloud | HTTP mit Schlüsselwort | `status.php` muss `installed:true` melden |
| Grafana, Prometheus, Homepage | HTTP | Health Endpunkte |
| Internet | Ping | `1.1.1.1` |

## 4.5 Nextcloud

Nextcloud läuft mit dem offiziellen Image, MariaDB 11.4 LTS als Datenbank, Redis für Cache und Dateisperren und einem eigenen Container für Hintergrund Jobs (Cron). Nur die App hängt am Netz `proxy`, Datenbank und Redis sind ausschließlich für Nextcloud erreichbar. Der Upload ist auf 10 GB pro Datei ausgelegt.

## 4.6 Homepage

Homepage ist die Startseite des Servers. Jede Kachel zeigt neben dem Link auch Live Werte, zum Beispiel geblockte Anfragen aus dem Pi-hole oder den Status aus Uptime Kuma. Die gesamte Konfiguration liegt als YAML im Repository. Passwörter werden über Umgebungsvariablen aus der `.env` eingesetzt.

## 4.7 Speicherbudget

| Stack | Grenze | Typisch |
|---|---|---|
| Pi-hole, Portainer | keine | ca. 150 MB |
| Nginx Proxy Manager | 256 MB | ca. 100 MB |
| Uptime Kuma | 256 MB | ca. 170 MB |
| Nextcloud (App, DB, Redis, Cron) | 768 + 512 + 128 + 128 MB | ca. 300 MB |
| Monitoring | 512 + 384 + 64 + 64 + 192 MB | ca. 550 MB |
| Homepage und Socket Proxy | 192 + 64 MB | ca. 130 MB |
| Jellyfin | 768 MB | ca. 190 MB |

# 5 Teil 3: Ausbau und Betrieb

Im dritten Teil ist aus dem Server ein System für den Alltag geworden. Dazu gehören ein eigener Streaming Dienst, nächtliche Backups, eine Firewall auf dem Host, eine Verwaltungsoberfläche im Browser und Alarme bei knappen Ressourcen. Außerdem wurde die Festplatte vollständig nutzbar gemacht und das erste große Systemupdate eingespielt. Alle Dateien liegen im Repository unter `docker/jellyfin`, `scripts` und `docker/monitoring/grafana/provisioning/alerting`.

## 5.1 Nextcloud aufgeräumt

Fotos vom iPhone kamen nie in der Cloud an. Das Log zeigte die Ursache: Die App Antivirus for files war aktiv, auf dem Server ist aber kein ClamAV installiert. Ohne Scanner lehnt Nextcloud jeden Upload ab (`No connection to anti virus`). Zusätzlich warf die DeepL App bei jedem Seitenaufruf einen Fehler, weil kein API Schlüssel eingetragen war.

Beide Apps wurden deaktiviert, ebenso weitere Zusatz Apps ohne Nutzen auf diesem Server: Recognize (Gesichtserkennung, zu schwer für 4 GB RAM und eine HDD), Universal File Viewer (überschneidet sich mit dem Standard Viewer), Claude Bot, Afterlogic, BigBlueButton, Analytics Sourcepack sowie drei Spiele. Deaktivieren löscht nichts und lässt sich mit `occ app:enable` jederzeit rückgängig machen.

## 5.2 Jellyfin und gemeinfreie Filme

Jellyfin ist ein eigener Streaming Dienst mit Bibliotheken für Filme und Serien, Covern und Weiterschauen, erreichbar als `https://jellyfin.home` im Browser, in der iPhone App und auf dem Fernseher. Wie alle Dienste hat der Container keinen Port am Host, Nginx Proxy Manager leitet über das Netz `proxy` weiter (Websockets eingeschaltet).

Die Medien liegen in `~/media` und sind nur lesend eingebunden. Die Grafikeinheit der CPU wird über `/dev/dri` durchgereicht, damit das Umwandeln von Videos nicht die vier alten Kerne auslastet. Der Container läuft als normaler Benutzer mit der Gruppe `render`.

Inhalte kommen bewusst nur aus legalen Quellen. Das Skript `scripts/archive-film.sh` lädt einen Film von archive.org: Es liest die Metadaten über die API, nimmt die größte MP4 Datei und legt sie als `Titel (Jahr)/Titel (Jahr).mp4` ab, so erkennt Jellyfin den Film automatisch. Fehlt im Eintrag eine Lizenzangabe, bricht das Skript ab, weil nicht alles auf archive.org gemeinfrei ist. Downloads von Streaming Seiten ohne Lizenz und das Umgehen von Kopierschutz (zum Beispiel bei Crunchyroll Downloads) sind ausgeschlossen.

## 5.3 Backups

Bisher gab es keine Sicherung. Ein Defekt der alten HDD oder ein missglücktes Update hätte Nextcloud, alle Einstellungen und die Dashboards gekostet. `scripts/backup.sh` läuft jetzt jede Nacht um 3:30 per cron und schreibt nach `~/backups/<Datum>`, die letzten drei Tage bleiben erhalten.

| Was | Wie |
|---|---|
| Nextcloud Datenbank | SQL Dump mit `mariadb-dump --single-transaction`, keine Kopie der laufenden Datenbankdateien |

| Was | Wie |
|---|---|
| Nextcloud Dateien | Volume als tar.gz im Wartungsmodus, Nextcloud bleibt erreichbar und Uptime Kuma meldet keinen Ausfall |
| Pi-hole, NPM, Portainer, Uptime Kuma, Grafana, Jellyfin | Container werden für wenige Sekunden gestoppt, damit die SQLite Datenbanken konsistent sind |
| Configs und CA | `~/homeserver` mit allen `.env` Dateien und die mkcert Zertifizierungsstelle |
| Bewusst nicht | Prometheus Daten, Jellyfin Cache und Redis, sie bauen sich von selbst wieder auf |

Der erste Lauf dauerte knapp zehn Minuten und ergab 1,5 GB. Geprüft wurden alle Archive mit `gunzip -t` und der Dump auf vollständige Tabellen. Falls ein Schritt fehlschlägt, startet ein `trap` die gestoppten Container trotzdem wieder und beendet den Wartungsmodus. Die Befehle zum Wiederherstellen stehen im Kopf des Skripts.

Grenze: Die Backups liegen auf derselben Festplatte. Sie schützen vor Fehlern und kaputten Updates, nicht vor einem Plattendefekt. Damit Grafana während des Stopps keinen Alarm auslöst, gibt es in Uptime Kuma ein tägliches Wartungsfenster von 3:25 bis 3:50.

## 5.4 Firewall mit ufw

Eine Bestandsaufnahme mit `ss -tuln` zeigte Dienste, die direkt am Host auf allen Schnittstellen lauschen. Wichtig dabei: Ports, die Docker veröffentlicht (53, 443, 81, 8080, 9443), schreibt Docker selbst in iptables, sie laufen an ufw vorbei. Diese sind aber schon in den Compose Dateien auf LAN und Tailscale Adresse gebunden. ufw schützt deshalb gezielt das, was am Host selbst läuft.

| Port | Dienst | Erlaubt aus |
|---|---|---|
| 22/tcp | SSH | LAN (Windows PC) und Tailnet |
| 80/tcp | CasaOS | LAN und Tailnet |
| 139, 445/tcp, 137, 138/udp | Samba | LAN und Tailnet |
| 9100/tcp | Node Exporter | nur Docker Netze (Prometheus) |
| 9090/tcp | Cockpit | nur Docker Netze (Nginx Proxy Manager) |
| 41641/udp | Tailscale WireGuard | überall, für direkte Verbindungen |
| alles andere | eingehend | gesperrt |

Die Regel für den Node Exporter ist leicht zu übersehen: Prometheus läuft im Container und fragt den Host über das Docker Netz ab. Ohne die Freigabe für `172.16.0.0/12` wären die Server Diagramme in Grafana leer geblieben.

Weil man sich mit einer Firewall über SSH selbst aussperren kann, hat `scripts/firewall.sh` eine Rückfallebene: Nach dem Einschalten startet ein Timer, der ufw nach zwei Minuten wieder abschaltet, wenn nicht im laufenden Fenster bestätigt wird, dass eine neue SSH Verbindung funktioniert. Zur Kontrolle wurden vom PC Verbindungen auf gesperrte Ports aufgebaut, sie erscheinen im Kernel Log als `[UFW BLOCK]` mit Absender und Zielport.

## 5.5 Cockpit

Cockpit ist die Weboberfläche von Ubuntu für den Server selbst: Übersicht über CPU, RAM und Platten, Protokolle, Dienste, Updates und ein Terminal im Browser, erreichbar als `https://cockpit.home`. Für ufw gibt es keine gepflegte Oberfläche, die geblockten Verbindungen lassen sich in Cockpit aber unter Protokolle mit dem Filter `UFW BLOCK` ansehen.

Installiert wurde mit `--no-install-recommends` ohne cockpit-networkmanager. Dieses Paket würde NetworkManager nachziehen, der Server läuft aber mit Netplan und systemd-networkd, ein zweiter Netzwerkdienst hätte die feste IP gefährdet. Cockpit lauscht nur auf der LAN Adresse, TLS endet am Proxy (`Origins` und `ProtocolHeader` in `/etc/cockpit/cockpit.conf`). Die Installation steht in `scripts/install-cockpit.sh`.

## 5.6 Grafana Alarme

Uptime Kuma meldet, wenn ein Dienst ausfällt. Grafana meldet jetzt zusätzlich, wenn der Server selbst an seine Grenzen kommt. Die Regeln sind als Provisioning Datei im Repository hinterlegt und werden beim Start automatisch angelegt. Benachrichtigt wird über dasselbe ntfy Topic, ntfy formatiert die Nachricht über sein eingebautes Template für Grafana.

| Alarm | Bedingung | Dauer |
|---|---|---|
| Festplatte fast voll | System Partition über 85 Prozent belegt | 10 Minuten |
| Arbeitsspeicher knapp | weniger als 10 Prozent RAM frei | 10 Minuten |
| CPU dauerhaft ausgelastet | über 90 Prozent | 15 Minuten |
| Swap stark belegt | über 80 Prozent, der Server lagert auf die HDD aus | 15 Minuten |
| Messquelle nicht erreichbar | Node Exporter, cAdvisor oder Pi-hole Exporter antwortet nicht (`up` ist 0) | 5 Minuten |

## 5.7 Festplatte erweitert

Die Installation von Ubuntu hatte von der 462 GB großen Partition nur 100 GB als logisches Volume angelegt, der Rest lag ungenutzt in der Volume Group. Mit `lvextend -r` wurde das Volume im laufenden Betrieb vergrößert, `-r` passt das ext4 Dateisystem direkt mit an. Ergebnis: 452 GB statt 98 GB, genug für Medien und Backups.

## 5.8 Erstes großes Systemupdate

Über Cockpit wurden rund 300 Pakete aktualisiert, darunter Docker, containerd, OpenSSH, systemd, libc und ein neuer Kernel. Während des Updates war SSH weg und alle Webdienste waren kurz nicht erreichbar, weil die Pakete ihre Dienste neu starten. Das Journal und `/var/log/apt/history.log` haben die Ursache eindeutig gezeigt.

Nach dem nötigen Neustart kam der Server nicht zurück. Ubuntu war sauber heruntergefahren, der alte PC blieb danach aber beim Neustart in der Firmware hängen. Der Monitor zeigte nichts, weil das HDMI Kabel erst nachträglich angesteckt wurde und das Mainboard das nicht erkennt. Ein harter Neustart mit

angeschlossenem Monitor löste das Problem. Danach liefen alle 16 Container, SSH, ufw, Tailscale und Cockpit automatisch wieder an.

Lehre daraus: Updates mit rund 15 Minuten Ausfall einplanen, erst nach dem Ende des Updates neu starten und bei diesem Rechner nach einem Neustart prüfen, ob er wirklich wieder da ist.

# 6 Sicherheit

| Maßnahme | Wirkung |
|---|---|
| Keine Portfreigaben, Zugriff nur über Tailscale | Kein Angriffspunkt aus dem Internet, kein Missbrauch für DNS Amplification |
| Nur der Proxy öffnet einen Web Port (443) | Eine einzige, kontrollierte Eingangstür |
| Port 53 und 443 gezielt auf LAN und Tailscale Adresse gebunden | Nicht aus fremden Netzen erreichbar |
| Proxy Verwaltung (81) nur im LAN | Proxy Konfiguration nicht aus dem Tailnet änderbar |
| HTTPS mit eigener CA | Verschlüsselt auch im Tailnet, ohne öffentliche Domain |
| Socket Proxy mit reinem Lesezugriff für Homepage | Kein Root Zugriff über das Dashboard |
| Nextcloud Datenbank und Redis nicht im Netz `proxy` | Nur für Nextcloud selbst erreichbar |
| Eigenes Pi-hole App Passwort | Das Admin Passwort liegt in keiner Datei |
| `mem_limit` auf allen neuen Containern | Kein Dienst kann den RAM aufbrauchen |
| SSH nur mit Ed25519 Schlüssel | Schutz gegen Brute Force |
| Device approval im Tailnet | Neue Geräte brauchen manuelle Freigabe |
| Passwörter nur in `.env`, per `.gitignore` ausgeschlossen | Keine Zugangsdaten im Repository |
| ufw auf dem Host, SSH, CasaOS und Samba nur aus LAN und Tailnet | Dienste am Host lauschen nicht mehr offen auf allen Schnittstellen |
| Node Exporter und Cockpit nur aus Docker Netzen erreichbar | Zugriff nur über Prometheus bzw. über HTTPS und den Proxy |
| Cockpit nur auf der LAN Adresse, Login mit Server Benutzer | Keine unverschlüsselte Verwaltung, nicht direkt aus dem Tailnet |
| Nächtliches Backup mit SQL Dump | Daten nach Fehlern oder kaputten Updates wiederherstellbar |
| Unnötige Nextcloud Apps deaktiviert | Weniger Angriffsfläche und weniger Fehlerquellen bei Updates |
| Grafana Alarme für Platte, RAM, CPU und Swap | Probleme fallen auf, bevor Dienste ausfallen |

# 7 Fehlersuche: Probleme und Lösungen

Jedes Problem wurde systematisch von unten nach oben eingegrenzt: Link, IP Adresse, Gateway, DNS, Dienst.

## 7.1 Grundsystem

| Symptom | Ursache | Lösung |
|---|---|---|
| Server per Ping und SSH nicht erreichbar | Alte statische IP `192.168.31.27` aus der Installation, falsches Subnetz | Netplan neu konfiguriert |
| `netplan apply`: expected mapping | Zweite Datei `01-net.yaml` mit kaputter Einrückung | Datei entfernt, YAML neu geschrieben |
| Keine IP per DHCP | Bei der ICS war kein Zieladapter ausgewählt | Ethernet als Heimnetzwerkverbindung gewählt |
| Destination Host Unreachable | Gateway als `192.168.137.137` statt `.1` eingetragen | Mit `ip route` gefunden und korrigiert |
| Port 53 nicht bindbar | `systemd-resolved` belegt Port 53 | `DNSStubListener=no` |
| Blocklisten laden nicht | Container ohne Namensauflösung beim Start | Fallback DNS im Container, Liste direkt von GitHub |
| Alle Tailscale Clients als `172.18.0.1` | Docker übersetzt die Quelladresse (NAT) | Verstanden und dokumentiert, Host Netzwerk als Option |

## 7.2 Erweiterung

| Symptom | Ursache | Lösung |
|---|---|---|
| Proxy startet nicht auf Port 80 | CasaOS belegt `*:80` auf dem Host | Proxy nur auf 443, Adressen immer mit `https://` |
| Container heißen `cadviso` und `node-exporte` | Beim Übertragen von Windows wurde aus `\r` ein `r`, sed hat jedes r am Zeilenende gelöscht | Dateien neu kopiert, Skripte nur noch als Datei per scp übertragen |
| Grafana zeigt keine Dashboards | Provisioning Ordner nach scp nur für den Besitzer lesbar | Ordnerrechte auf 755 gesetzt |
| Serverlast 7,5, Grafana Fehler 502 | cAdvisor scannt die Belegung jedes Containers, auf der HDD minutenlang | Disk Metriken abgeschaltet, Grafana mehr RAM |
| Uptime Kuma meldet 400 für Homepage | Homepage erlaubt nur bekannte Hostnamen | Internen Namen `homepage:3000` erlaubt |
| Pi-hole Passwort nicht auslesbar | Passwort in der Oberfläche gesetzt, nur als Hash gespeichert | Eigenes App Passwort über die Pi-hole API |
| PC hat plötzlich kein Internet | Tailscale am PC übernahm das DNS, war aber noch nicht freigegeben | `--accept-dns=false`, `.home` per hosts Datei |
| PC verbindet sich nicht mehr mit der FritzBox | Altes WPA3 Profil passte nicht zur geänderten WLAN Sicherheit | WLAN Profil gelöscht und neu verbunden |

## 7.3 Ausbau und Betrieb

| Symptom | Ursache | Lösung |
|---|---|---|
| Fotos vom iPhone kommen nicht an | Antivirus App aktiv, aber kein ClamAV installiert, jeder Upload wird abgelehnt | App deaktiviert, Foto erneut hochgeladen |
| Fotos am PC nicht sichtbar | Am PC als `admin` angemeldet, hochgeladen wurde als eigener Benutzer | Mit dem eigenen Benutzer anmelden |
| Fehler bei jedem Seitenaufruf in Nextcloud | DeepL App ohne API Schlüssel | App deaktiviert |
| Portainer Kachel zeigt API Fehler | Access Token gelöscht, Portainer antwortet mit 401 | Neuen Token angelegt und eingetragen |
| `jellyfin.home` nicht gefunden | PC nutzt für `.home` die hosts Datei, der neue Name fehlte | Eintrag im Terminal als Administrator ergänzt |
| Jellyfin Monitor: wrong version number | HTTPS auf den HTTP Port 8096 | `http://jellyfin:8096/health` mit Keyword `Healthy` |
| Cockpit Monitor bleibt grau | Kuma Container kennt `.home` Namen und eigene CA nicht | Direkt `http://192.168.137.10:9090/ping` prüfen |
| Grafana nachts kurz offline | Backup stoppt Grafana für einen konsistenten Stand | Wartungsfenster in Uptime Kuma |
| SSH plötzlich abgelehnt, Webdienste kurz weg | Systemupdate über Cockpit startet OpenSSH und Docker neu | Update abwarten, `ssh.socket` neu starten |
| Server nach Neustart weg, Monitor ohne Bild | Mainboard hängt nach sauberem Shutdown, HDMI ohne Hotplug | Harter Neustart mit angestecktem Monitor |
| `resize2fs`: Device or resource busy | `lvextend` zweimal ausgeführt, die erste Vergrößerung lief noch | `resize2fs` erneut ausgeführt |
| Grafana Alarme werden nicht geladen | Grafana 13 akzeptiert `execErrState: Error` nicht mehr | Wert auf `Alerting` geändert |
| Namen wie `Serve` und `cadviso` in Dateien | `sed "s/\r$//"` über PowerShell und SSH, die Anführungszeichen gingen verloren und `\r` wurde zu `r` | Dateien ohne sed neu kopiert, Prüfsummen verglichen, Zeilenenden per `.gitattributes` |

# 8 Ergebnis

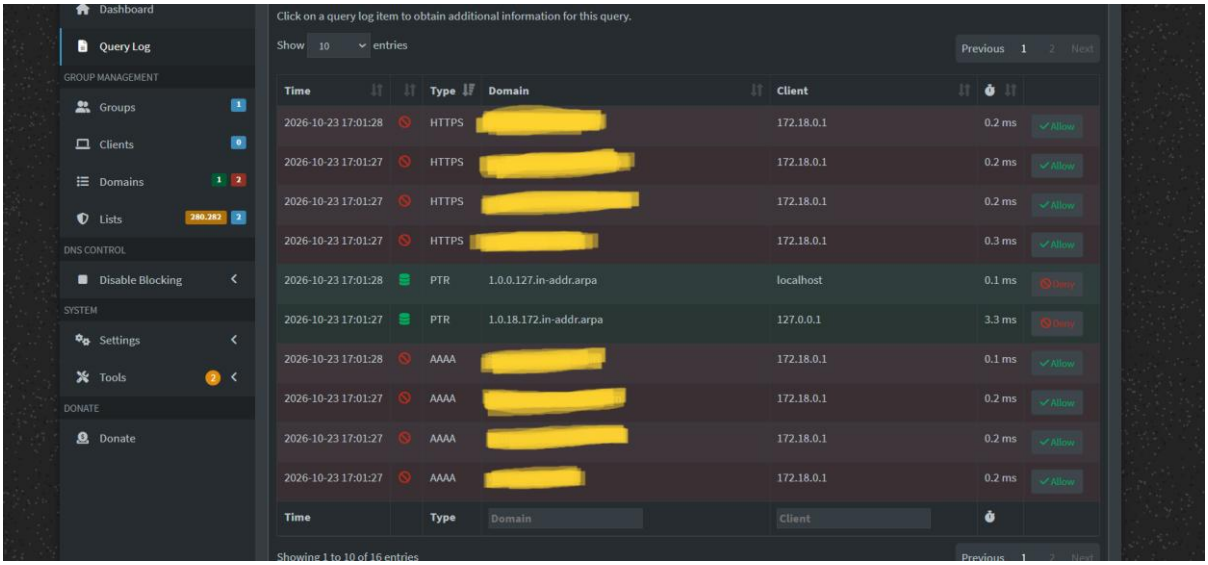
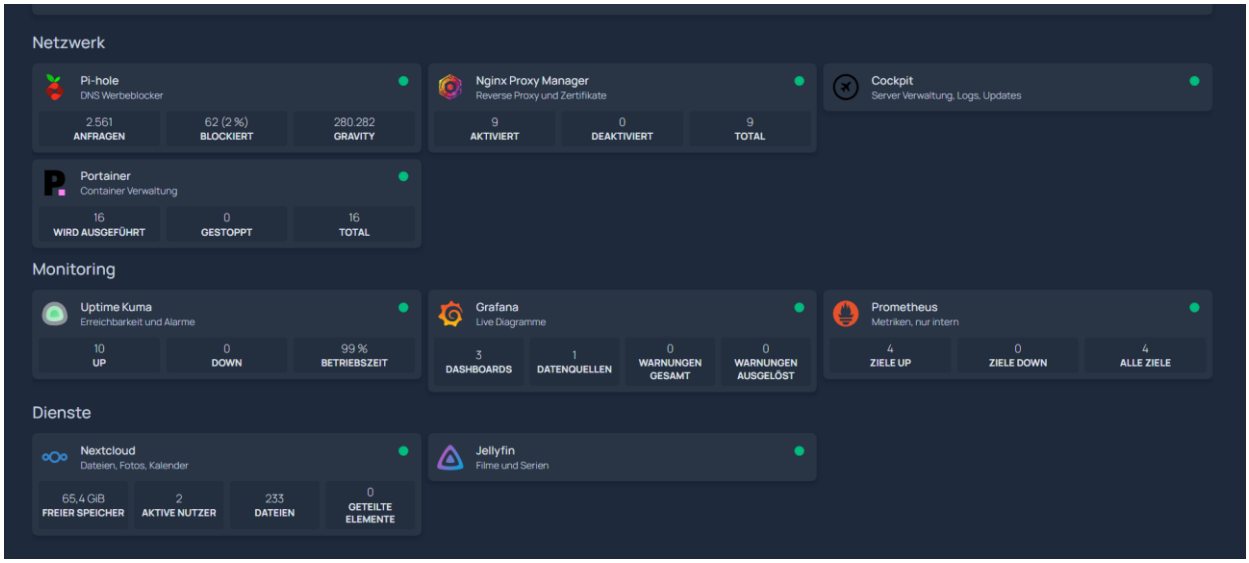



*Abbildung 3: Geblockte Anfragen im Query Log des Pi-hole*




*Startseite (Homepage) mit Live Werten aller Dienste*




*Abbildung 5: Pi-hole Statistik in Grafana*

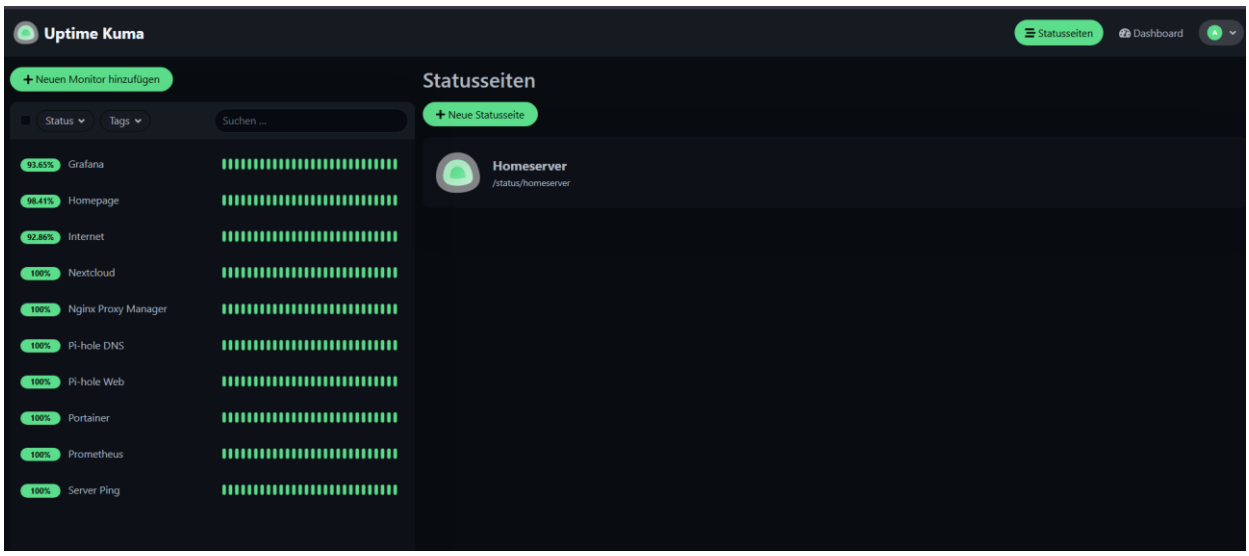



*Abbildung 6: Uptime Kuma mit allen zehn Monitoren*

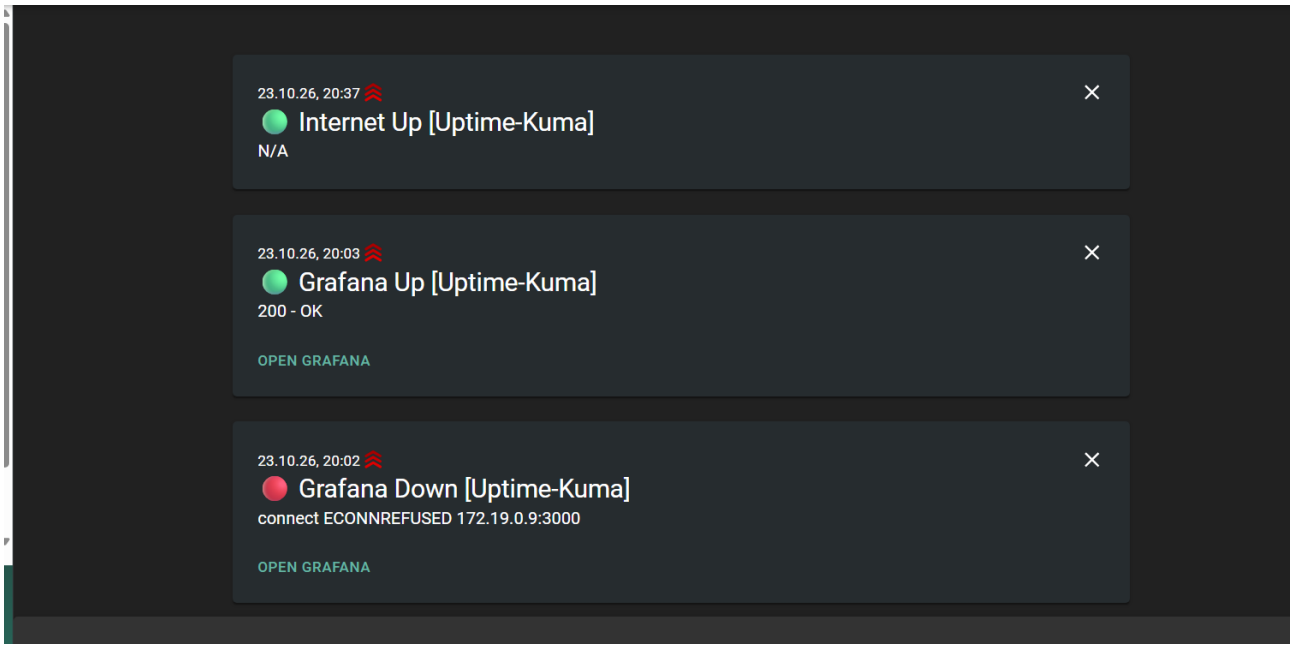



*Abbildung 7: Ntfy Push-Benachrichtigungen mit UP/DOWN*

Werbe und Tracking Anfragen vom Handy werden über das Tailnet zuverlässig geblockt. Alle Dienste sind über HTTPS mit sprechenden Namen erreichbar, werden minütlich überwacht und melden Ausfälle aufs Handy. Prometheus erreicht alle vier Ziele, Grafana zeigt die drei Dashboards, und alle zehn Monitore in Uptime Kuma sind grün.

Der Server ist komplett per SSH und Browser verwaltbar und von außen nicht angreifbar.

Mit dem dritten Teil kommen Filme über Jellyfin, nächtliche Backups, eine Host Firewall, Cockpit und Alarme bei knappen Ressourcen dazu. Die Festplatte ist vollständig nutzbar und das System ist auf aktuellem Stand.

# 9 Was ich gelernt habe

- Wie die Windows Internetverbindungsfreigabe intern als kleiner Router mit DHCP und NAT arbeitet.
- Wie man Netzwerkfehler mit `ip a`, `ip route` und `ping` Schicht für Schicht eingrenzt.
- Wie Docker Bridge Netzwerke und Portbindungen funktionieren und warum dadurch Quelladressen verloren gehen.
- Wie ein Mesh VPN ohne Portfreigaben durch CGNAT kommt.
- Wie ein Reverse Proxy mehrere Dienste hinter einer Adresse bündelt und wie Zertifikate und eine eigene CA zusammenspielen.
- Wie Prometheus Metriken einsammelt, Grafana sie darstellt und warum Monitoring auf schwacher Hardware selbst zur Last werden kann.
- Dass ein VPN, das das DNS übernimmt, einem PC komplett das Internet nehmen kann, und wie man das sauber trennt.
- Warum von Docker veröffentlichte Ports an ufw vorbeilaufen und was eine Firewall auf dem Host trotzdem schützt.
- Wie man Datenbanken konsistent sichert, mit SQL Dump und Wartungsmodus statt Kopien laufender Dateien.
- Wie LVM aufgebaut ist und wie man ein logisches Volume im laufenden Betrieb vergrößert.
- Dass Updates Dienste neu starten, ein neuer Kernel einen Neustart braucht und alte Hardware dabei hängen bleiben kann.
- Dass Anführungszeichen auf dem Weg von PowerShell über SSH nach bash verloren gehen können, und warum man kopierte Dateien mit Prüfsummen kontrolliert.